```
# train.py
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras import layers, models
import json

IMAGE_SIZE = (224, 224)
batch_size = 32

train_path = "C:\\Users\\HP\\Downloads\\Bone Break Classification\\Bone Break Classification\\Train"
test_path = "C:\\Users\\HP\\Downloads\\Bone Break Classification\\Bone Break Classification\\Test"

datagen = ImageDataGenerator(rescale=1./255)

train_ds = datagen.flow_from_directory(
    train_path,
    target_size=IMAGE_SIZE,
    batch_size=batch_size,
    class_mode='categorical'
)

val_ds = datagen.flow_from_directory(
    test_path,
    target_size=IMAGE_SIZE,
    batch_size=batch_size,
    class_mode='categorical'
)

model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(224, 224, 3)),
    layers.MaxPooling2D(),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D(),
```

```
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(train_ds.num_classes, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(train_ds, epochs=10, validation_data=val_ds)

model.save("bone_model.h5")
print("Model saved successfully")

with open("labels.json", "w") as f:
    json.dump(train_ds.class_indices, f)

print("Labels saved successfully")

# test.py
import tkinter as tk
from tkinter import filedialog
from PIL import Image, ImageTk
import numpy as np
import tensorflow as tf
import json
from tensorflow.keras.preprocessing import image

model = tf.keras.models.load_model("bone_model.h5")

with open("labels.json", "r") as f:
    class_indices = json.load(f)
```

```
labels = list(class_indices.keys())

def predict_image(img_path):
    img = image.load_img(img_path, target_size=(224, 224))
    img = image.img_to_array(img)
    img = img / 255.0
    img = np.expand_dims(img, axis=0)

    result = model.predict(img)
    idx = np.argmax(result)

    return labels[idx]

def open_image():
    global img_path, panel

    img_path = filedialog.askopenfilename(
        filetypes=[("Image files", "*.jpg *.jpeg *.png")]
    )

    img = Image.open(img_path)
    img = img.resize((300, 300))
    img_tk = ImageTk.PhotoImage(img)

    panel.config(image=img_tk)
    panel.image = img_tk
    result.config(text="")

def classify():
    if img_path:
        label = predict_image(img_path)
        result.config(text=f"Prediction: {label}")

root = tk.Tk()
root.title("Bone Fracture Detection")
```

```
root.geometry("400x500")

panel = tk.Label(root)
panel.pack(pady=10)

btn_load = tk.Button(root, text="Load X-ray Image", command=open_image)
btn_load.pack(pady=10)

btn_predict = tk.Button(root, text="Detect Fracture", command=classify)
btn_predict.pack(pady=10)

result = tk.Label(root, text="", font=("Arial", 20))
result.pack(pady=20)
img_path = ""
root.mainloop()
```

Outputs:

```
Epoch 1/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 51s 1s/step - accuracy: 0.1417 - loss: 2.3722 - val_accuracy: 0.1786 - val_loss: 2.2124
Epoch 2/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 28s 766ms/step - accuracy: 0.2471 - loss: 2.1049 - val_accuracy: 0.4143 - val_loss: 1.8186
Epoch 3/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 29s 803ms/step - accuracy: 0.4216 - loss: 1.6691 - val_accuracy: 0.6071 - val_loss: 1.2473
Epoch 4/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 303s 9s/step - accuracy: 0.6014 - loss: 1.1703 - val_accuracy: 0.7786 - val_loss: 0.8186
Epoch 5/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 31s 845ms/step - accuracy: 0.7733 - loss: 0.7683 - val_accuracy: 0.8357 - val_loss: 0.4594
Epoch 6/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 30s 827ms/step - accuracy: 0.8707 - loss: 0.4663 - val_accuracy: 0.9571 - val_loss: 0.2261
Epoch 7/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 41s 1s/step - accuracy: 0.9229 - loss: 0.3456 - val_accuracy: 0.9643 - val_loss: 0.1594
Epoch 8/10
```